

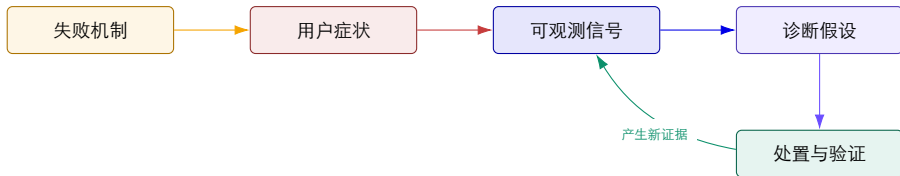
面向 AI 原生系统的智能运维

第3讲 | 智能监控与可观测性工程（1/3）

从 Metrics、Logs、Traces 到可供人和智能体使用的证据系统

陈鹏飞 | 中山大学

与上一讲的连接：知道系统如何失败，还需要找到证据



上一讲 识别推理、RAG、Agent、GPU、数据与安全的失败模式。

本讲 设计指标、埋点、采集、存储与关联，让内部状态可以被推断。

后续课程 在日志、追踪、异常检测和根因定位中消费这些证据。

本讲核心问题

1. 区分监控与可观测性，解释“从输出推断内部状态”的工程含义；
2. 说明 Metrics、Logs、Traces 的优势、局限和关联方式；
3. 为 Web、GPU、训练、推理、RAG 与 Agent 设计可操作的指标；
4. 使用 Prometheus 数据模型和 PromQL 表达常见运维问题；
5. 解释 OpenTelemetry API、SDK、Collector 与后端之间的职责；
6. 分析时序数据库的基数、保留期、降采样、WAL 与远程存储权衡；
7. 把可观测数据映射到告警治理、故障诊断、容量分析和智能体工具调用。

学习结果不是“会画一个 Dashboard”，而是能把一个运维问题转换成可采集、可查询、可验证的证据。

开场事故：TTFT 告警触发了，但究竟哪里慢？

用户与业务现象

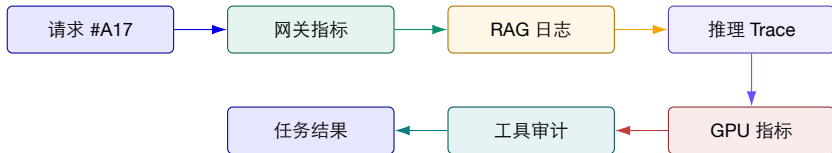
- ▶ 智能客服首字等待从 1.4s 升到 8.7s；
- ▶ 总响应时间上升，答案质量暂未明显下降；
- ▶ 同一时段营销活动带来 2.3 倍请求量；
- ▶ 值班人员只收到“P95 延迟过高”一条告警。

可能的内部原因

- ▶ API 网关排队、RAG 检索慢或推理队列堵塞；
- ▶ 长 Prompt 增多导致 Prefill 时间和 KV 占用上升；
- ▶ 模型副本扩容慢，或某个 GPU 节点降频；
- ▶ 下游内容审核或工具调用出现尾延迟。

关键问题 一个“延迟指标”只能告诉我们发生了什么，不能自动告诉我们为什么发生。

证据散落在不同系统：没有关联，就只能靠猜



如果没有统一 **ID** 无法确认高显存节点是否处理了这个慢请求。

如果时间不一致 日志与指标的先后关系可能被错误解读。

如果语义不一致 同名 **latency** 可能分别表示队列、模型或端到端延迟。

采集成本算例：多加一个标签会增加多少序列？

假设每个组合都被实际观测：12 个服务、4 个区域、3 类状态；每个组合暴露一个普通计数器。

设计	组合数	活跃序列	查询能力
基础标签	$12 \times 4 \times 3$	144	按服务、区域、状态聚合
加入 50 个租户	144×50	7,200	可按租户分析
再加请求 ID	随请求持续增长	无法用固定乘数封顶	请求级细节应进入 Trace
15 秒采集	每天 5,760 次	41,472,000 样本/天	仅计算 7,200 条序列

这是普通计数器的样本量估算；直方图、副本、索引与压缩的成本需另算。

推导与判断 删除租户标签能降成本，却失去租户维度；应按确实需要回答的查询保留维度，不能只追求数据量少。

算例使用假设数据。

本讲路线图

1. 从监控走向可观测性：建立证据与推理语言；
2. 理解 Metrics、Logs、Traces 三类核心信号；
3. 设计指标模型，控制标签基数并覆盖 AI 工作负载；
4. 使用 Prometheus 采集、查询和预计算时序数据；
5. 使用 OpenTelemetry 统一埋点、上下文与传输；
6. 设计时序存储与企业级 AIOps 数据采集与存储；
7. 通过 TELLER 连接请求级信号与 GPU 跨层诊断研究。

可观测问题 → 信号设计 → 采集与存储 → 关联与推理 → 决策与验证

一、从监控到可观测性

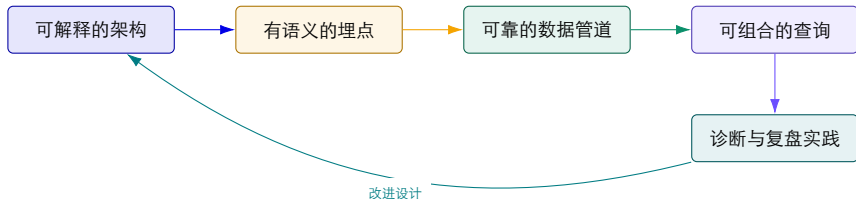
监控回答预先定义的问题；可观测性帮助我们探索尚未预见的问题。

监控与可观测性：不是替代关系，而是问题空间不同

维度	监控 Monitoring	可观测性 Observability
出发点	已知组件、阈值与失败模式	从外部输出推断系统内部状态
典型问题	CPU 是否超过 90%？服务是否存活？	为什么只有长上下文租户的 TTFT 上升？
主要手段	固定 Dashboard、规则与告警	指标、日志、追踪、上下文和交互式查询
价值	快速发现已知风险	缩小未知问题的假设空间
局限	规则之外的问题容易漏检	埋点、语义、存储和人员能力成本更高

关系 好的可观测性让监控规则更有依据；好的监控把探索所得的知识固化为可重复检测。

“可观测”是一种系统属性，不是采购一个平台



软件侧 暴露边界、状态、版本、依赖和业务结果。

平台侧 统一采集、时间、身份、保留与访问控制。

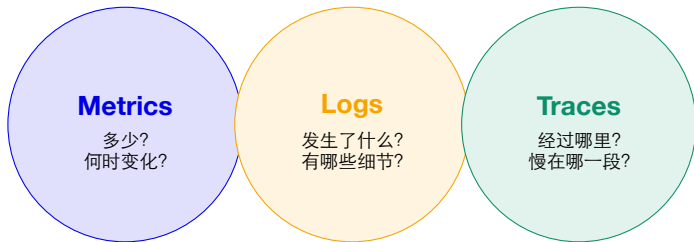
组织侧 值班、Runbook、复盘和反馈推动埋点演化。

三类核心信号：同一系统的三种观察角度

Metrics 把大量事件聚合为随时间变化的数值，适合发现趋势、阈值和容量。

Logs 记录离散事件及其上下文，适合解释细节、错误与审计。

Traces 记录一次请求跨组件的路径和耗时，适合定位依赖与传播。



Metrics: 低成本地观察群体行为

擅长回答

- ▶ 请求率、错误率、延迟分位数如何变化?
- ▶ GPU、队列、缓存和连接池是否接近容量边界?
- ▶ 哪个模型、区域或租户的总体表现异常?
- ▶ 是否违反 SLO, 是否需要扩缩容?

不擅长回答

- ▶ 某个具体请求为什么失败?
- ▶ 一条异常调用经过了哪些服务?
- ▶ 错误堆栈、Prompt 或工具参数是什么?
- ▶ 高基数明细不能无限塞进标签。

指标是一种有损压缩: 它牺牲单事件细节, 换取低成本、长时间范围和快速聚合。

Logs: 保留事件语义与上下文

擅长回答

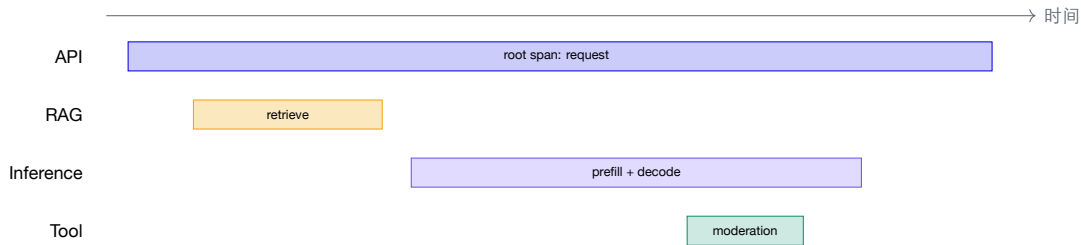
- ▶ 哪个版本、配置、用户动作触发了错误？
- ▶ 工具调用参数、返回码和安全决策是什么？
- ▶ 模型加载、分片恢复、重试发生了什么？
- ▶ 是否需要审计或重放关键事件？

工程约束

- ▶ 自由文本难解析，应优先结构化字段；
- ▶ 高频日志产生显著存储与检索成本；
- ▶ Prompt、PII、密钥和内部推理需脱敏；
- ▶ 没有 request/trace ID 就难与其他信号关联。

原则 日志不是“把所有变量打印出来”，而是记录能解释状态转换和决策边界的事件。

Traces: 保留一次请求的因果路径

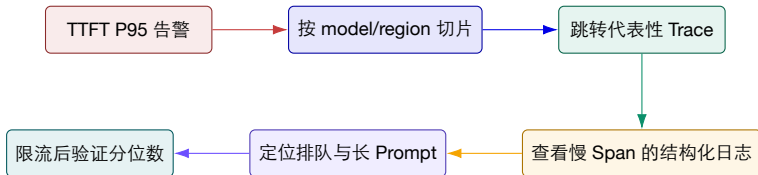


Span 操作的时间、状态、属性与事件。

Trace 父子 Span 组成的端到端请求。

代价 全量追踪昂贵，需采样和脱敏。

三支柱的价值来自“关联”，而不是三个独立页面



一条理想路径 指标负责发现群体异常，Trace 缩小到一条路径，日志补充决策细节，再回到指标验证处置是否对总体有效。

上下文传播：让一次请求在跨进程后仍是“同一个故事”



trace_id: 连接整条请求路径 span_id: 标识当前操作
request_id / session_id: 业务关联 baggage: 谨慎传播的业务上下文

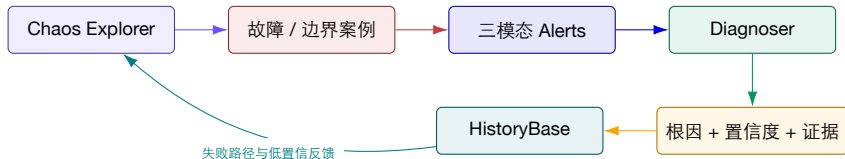
传播失败会制造“断链”；传播过多则可能泄露敏感数据或增加网络开销。

分层埋点：每一层都回答不同的诊断问题

层级	代表性信号	能回答的问题
业务结果 Agent/RAG 模型服务 平台服务 基础设施	任务成功率、转人工率、有效回答率 规划步数、工具成功率、检索命中、引用新鲜度 TTFT、TPOT、tokens/s、队列、批大小、KV 使用 RED 指标、依赖 Trace、错误日志 GPU/CPU/网络/磁盘/温度/功耗	系统是否真正完成用户目标？ 是计划、知识还是工具出了问题？ 推理慢在排队、Prefill 还是 Decode？ 哪个服务或调用边界传播了故障？ 是否存在资源饱和、硬件或拓扑问题？

连接规则 上层信号定义“是否有价值”，下层信号解释“为什么做不到”；不能只观察其中一层。

OpsLoop: 为什么把 Metrics、Traces、Logs 组织为三模态证据



Metrics 快速发现群体偏离、定位异常窗口与候选对象。

Traces 还原传播路径和慢在哪里，区分边界与下游。

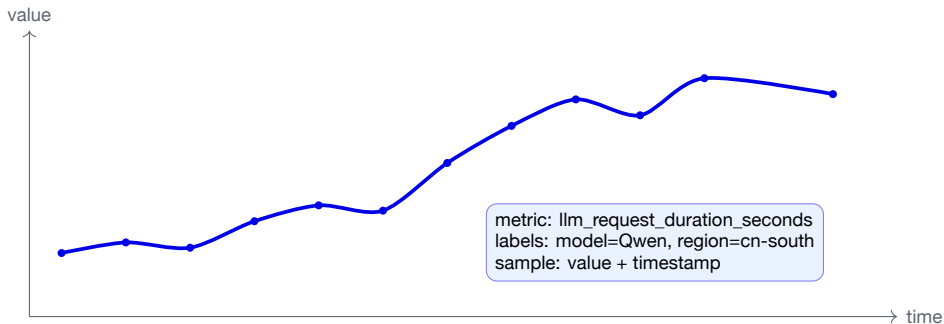
Logs 补充错误语义、配置、状态转换和工具返回。

这不是为了“输入更多 token”，而是让 Diagnoser 能用独立来源互相支持或反驳，并把失败案例用于改进下一轮实验。

二、指标体系与数据模型

指标不是随手暴露的数字，而是对系统状态的可查询编码。

时间序列：一个数值加上它属于谁、发生在何时



定义 在 Prometheus 中，一条时间序列由“指标名 + 一组标签”唯一标识；任何标签值变化都会形成新序列。

选择指标类型，本质上是在声明“数值如何变化”

类型	变化语义	典型示例	常见误用
Counter Gauge	只增不减，重启可归零 可上升也可下降	请求数、错误数、token 总数 队列长度、GPU 显存、在线副本	直接告警累计值，不计算速率 用于累计事件，导致重启后语义混乱
Histogram	将观测值计入桶并记录 count/sum	延迟、批大小、Prompt 长度	桶边界不合理，无法估计目标分位数
Summary	客户端计算分位数并记录 count/sum	单实例局部分位数	跨实例聚合 quantile，数学上不成立

选择原则 先问“这个量是累计事件、当前状态，还是一组观测值的分布？”再决定类型。

Counter: 观察事件发生的速率, 而不是盯着累计值

12,480

llm_requests_total 累计值

问题 服务运行越久累计值越大; “12480 是否异常” 没有统一答案。

把 Counter 转换为可操作信息

- ▶ `rate(...[5m])`: 每秒平均增长速率;
- ▶ `increase(...[1h])`: 一小时内增长量;
- ▶ `error rate = error rate / all request rate`;
- ▶ 重启导致归零, PromQL 会处理 Counter reset。

```
sum(rate(llm_requests_total{status="error"}[5m])) / sum(rate(llm_requests_total[5m]))
```

Gauge: 表示某一时刻可增可减的状态

37

inference_waiting_requests

91%

gpu_memory_used_ratio

12

model_replicas_ready

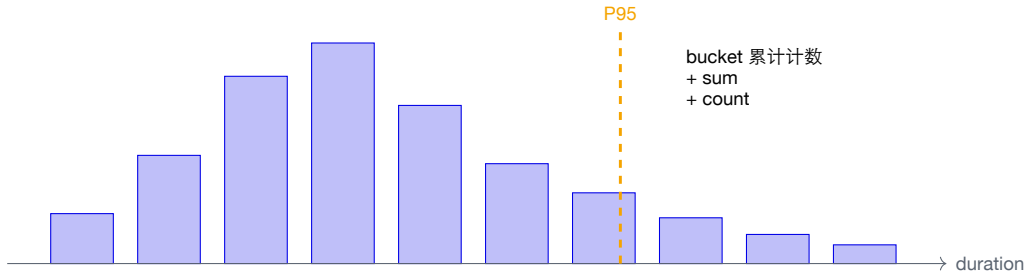
适合

- ▶ 队列、连接、线程、在线副本；
- ▶ 内存、磁盘、温度、功率；
- ▶ 当前并发、当前批大小；
- ▶ 最近一次成功时间戳。

分析方式

- ▶ 当前值与容量上限的距离；
- ▶ `max_over_time` 捕捉峰值；
- ▶ `deriv` 或窗口变化观察趋势；
- ▶ 与请求率联动判断饱和还是泄漏。

Histogram: 保留分布, 才能讨论尾延迟



桶 记录“小于等于边界”
的累计观测次数。

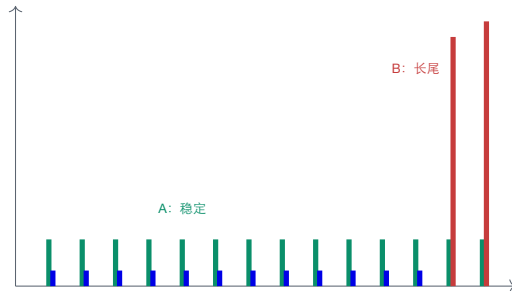
可聚合 不同实例使用相
同桶时, 可先求和再估计
分位数。

要设计 桶过粗看不清
SLO, 过密则增加序列与
成本。

为什么平均值会掩盖用户痛苦？

两个系统平均值相同

- ▶ 系统 A: 100 个请求全部约 1 秒;
- ▶ 系统 B: 95 个请求 0.2 秒, 5 个请求 16.2 秒;
- ▶ 两者平均值都约为 1 秒;
- ▶ 但系统 B 的 P95/P99 与用户体验完全不同。



AI 推理易出现长尾：输入长度、批处理、排队、抢占和工具调用会扩大请求差异。

Histogram 桶应围绕服务目标与物理机制设计

TTFT 目标示例

- ▶ SLO: 95% 请求 $TTFT \leq 2s$;
- ▶ 建议边界: 0.1, 0.25, 0.5, 1, 2, 4, 8, 16s;
- ▶ 2s 附近必须有桶边界, 才能直接计算达标率;
- ▶ 还需覆盖超长排队, 避免所有异常都落入 +Inf。

Prompt 长度示例

- ▶ 结合上下文窗口和调度策略分桶;
- ▶ 如 128, 512, 1k, 2k, 4k, 8k, 16k tokens;
- ▶ 用于解释 Prefill 时间、KV 使用与拒绝率;
- ▶ 边界变化会影响历史可比性, 应版本化。

经验 桶不是越多越好; 每增加一个桶, 都会按标签组合增加新的时间序列。

RED 与 USE：从服务体验和资源瓶颈两侧夹击

RED：面向服务请求

- ▶ **Rate**：每秒处理多少请求？
- ▶ **Errors**：失败或不合格请求占比？
- ▶ **Duration**：延迟分布如何？
- ▶ 适合 API、RAG、模型和工具服务。

USE：面向资源

- ▶ **Utilization**：资源忙碌程度？
- ▶ **Saturation**：排队或等待程度？
- ▶ **Errors**：硬件/驱动/网络错误？
- ▶ 适合 CPU、GPU、磁盘、网络 and 线程池。

组合诊断 请求延迟上升 + GPU 利用率低 + waiting 队列高，可能不是算力不足，而是调度、数据传输或依赖阻塞。

AI 原生系统的指标不止“基础设施 + HTTP”

业务与任务：解决率、人工接管率、任务成功率、收入/成本影响

Agent/RAG：步骤数、工具成功率、检索质量、引用新鲜度、循环次数

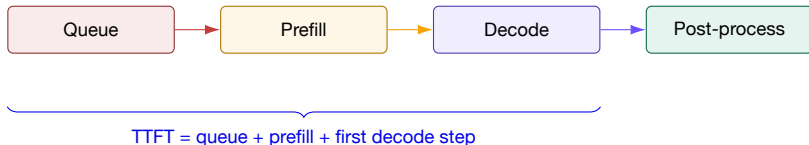
模型：TTFT、TPOT、tokens/s、队列、批处理、KV Cache、质量代理

服务：请求率、错误率、延迟、依赖、重试、限流

资源：GPU/CPU、显存、网络、磁盘、功耗

底层健康是必要条件，不是任务成功的充分条件；指标体系必须一直连接到业务结果。

LLM 推理：把端到端延迟拆成可行动的机制指标



指标	解释	常见关联
TTFT 平均 TPOT / ITL tokens/s KV Cache usage cost/request	从请求到首 token 首 token 后平均每 token 时间 / 相邻 token 间隔 单请求或实例吞吐 解码状态占用 token、GPU 时间与外部调用成本	waiting queue、input tokens、Prefill 吞吐 Decode 批次、GPU 利用率、抢占 并发、批大小、模型规格、量化 上下文长度、并发、block 分配、抢占 路由、重试、缓存命中、Agent 步数

训练与 GPU：同时观察进度、效率、正确性和硬件健康

维度	指标示例	解释与动作
训练进度	step、epoch、samples/s、tokens/s	是否推进？是否因数据或同步停滞？
模型行为	loss、梯度范数、学习率、溢出次数	是否收敛？是否出现数值不稳定？
并行效率	step time、all-reduce time、straggler ratio	慢在计算、通信还是数据加载？
GPU 使用	utilization、memory、SM occupancy、tensor core	算力是否充分利用？是否被显存限制？
硬件健康	Xid、ECC、温度、功耗、NVLink/PCIe errors	是否需要隔离节点、降载或更换硬件？

例子 GPU 利用率 99% 不一定代表高效率：若 step time 上升、tokens/s 下降，可能在做无效重算或被慢节点拖累。

论文案例：ByteRobust 把训练可观测性放进每个 Pod 的数据面

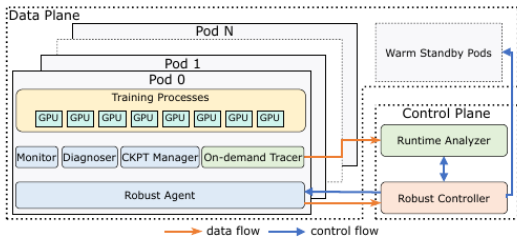


Figure 4. Architecture of ByteRobust.

不是只放一个 Monitor

- ▶ Monitor: 持续采集训练和系统信号;
- ▶ Diagnoser: 训练暂停后运行分层诊断;
- ▶ On-demand Tracer: 按需抓取进程栈;
- ▶ CKPT Manager: 提供恢复状态与检查点;
- ▶ Runtime Analyzer: 跨 Pod 聚合并识别异常。

工程启示 采用“实时低成本监控 + 异常后按需深追踪”，避免对大规模训练持续开启高开销诊断。

ByteRobust 的三类训练证据：指标、日志/退出码与系统事件

论文中的数据类别	代表信号	对诊断的作用
工作负载训练指标	loss、gradient norm、MFU、训练进度	5 倍突增、NaN、MFU 下降等用于发现训练行为异常
stdout/stderr 与退出码 CUDA/RDMA/主机/存储	Python 异常、NCCL/CUDA 报错、进程退出 RDMA 流量、TensorCore 利用、Xid、磁盘与主机事件	提供用户代码或运行时错误的语义线索 识别通信停滞、硬件和基础设施故障
按需进程栈	各 Rank 的调用栈与并行组关系	日志沉默或整体同步变慢时定位离群机器/进程

实时阶段 用便宜、连续的信号快速发现已知异常。

停机诊断阶段 只在需要时采集高维证据，换取更深定位。

论文案例：Minder 说明“更多指标”不等于“更好检测”

Minder 的设计选择

1. 先对每台机器的时序数据对齐、归一化和去噪；
2. 为不同指标分别训练模型，而非把全部指标硬拼在一起；
3. 按指标对故障的敏感度确定检查顺序；
4. 比较机器间相似性，并用跨窗口持续性过滤瞬态抖动。

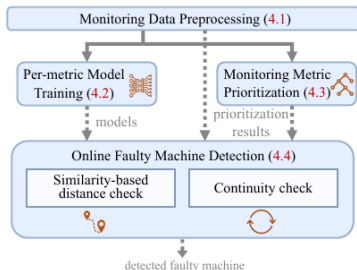


Figure 5: System architecture of Minder

与本讲的连接 指标选择要考虑可区分性、噪声和持续性；采集更多弱信号可能降低精度并增加查询成本。

命名与单位：让跨团队查询不需要猜

推荐约定

- ▶ 使用小写 `snake_case`;
- ▶ 名称包含领域与被测对象;
- ▶ 基础单位进入后缀: `_seconds`、`_bytes`;
- ▶ Counter 以 `_total` 结尾;
- ▶ 在文档中定义范围、单位、类型和标签。

从模糊到清晰

- ▶ latency $\Rightarrow$
`llm_request_duration_seconds`;
- ▶ memory $\Rightarrow$ `gpu_memory_used_bytes`;
- ▶ errors $\Rightarrow$
`tool_calls_total{status="error"}`;
- ▶ tokens $\Rightarrow$ 区分 `prompt/completion` 与 `total`。

命名是数据格式约定。指标改名、换单位或改语义，应像 API 变更一样管理。

标签设计：只放“有限、稳定、可聚合”的维度

通常适合作为标签

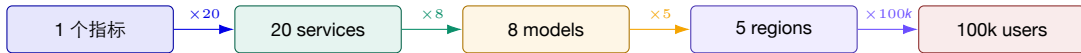
- ▶ service、region、cluster、namespace；
- ▶ model、model_version、route；
- ▶ status、error_type、tool_name；
- ▶ tenant_tier、request_class；
- ▶ 值集合可枚举且变化受控。

通常不适合作为标签

- ▶ user_id、request_id、trace_id；
- ▶ 完整 URL、Prompt、异常堆栈；
- ▶ 时间戳、随机 ID、文件路径；
- ▶ 任意工具参数或文档 ID；
- ▶ 应放入日志、Trace 或专用明细存储。

判断题 “我是否真的会按这个维度做聚合、告警或容量决策？” 如果不会，它很可能不应成为标签。

基数爆炸：每个标签的取值会相乘



800

不含 user_id 的序列数

80,000,000

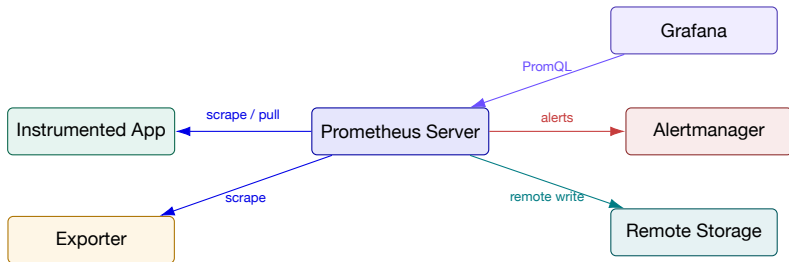
加入 user_id 后的理论序列数

Histogram 还要乘以 bucket 数；再叠加实例、副本、状态等维度，成本会进一步放大。

三、Prometheus：采集与查询指标

理解它的数据流，才能正确设计暴露、抓取、查询、预计算和告警。

Prometheus 架构：从目标抓取到查询与告警

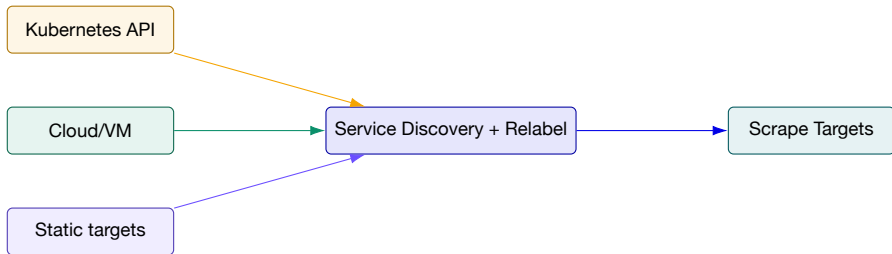


发现 通过静态配置、Kubernetes、云服务或其他机制发现目标。

抓取与存储 周期性读取 /metrics，写入本地 TSDB。

消费 PromQL、Dashboard、记录规则、告警与远程分析。

服务发现与 **Exporter**: 把异构目标变成统一暴露格式



原生埋点 应用直接使用客户端库，暴露业务和运行时指标。

Exporter 为无法修改的系统翻译指标，如 node、数据库、NVIDIA DCGM exporter。

Relabeling 应服务于目标选择与有限标签治理，不应成为不可维护的业务逻辑中心。

/metrics 暴露的是当前可抓取状态

```
# HELP llm_requests_total LLM requests
# TYPE llm_requests_total counter
llm_requests_total{model="qwen",status="ok"} 18432
# HELP inference_waiting_requests Queue size
# TYPE inference_waiting_requests gauge
inference_waiting_requests{model="qwen"} 37
```

Prometheus 定期采样暴露值；它不是业务事件消息队列。

暴露端责任

- ▶ 正确维护 Counter/Gauge/Histogram 状态；
- ▶ 使用稳定命名、单位和有限标签；
- ▶ /metrics 响应要快且可用；
- ▶ 不泄露用户数据、密钥和 Prompt；
- ▶ 处理多进程与重启语义。

Python 埋点示例：请求计数与延迟分布

```
from prometheus_client import Counter, Histogram

REQUESTS = Counter("llm_requests_total", "LLM requests", ["model", "status"])
LATENCY = Histogram("llm_request_duration_seconds", "Latency",
                    ["model"], buckets=(.1,.25,.5,1,2,4,8,16))

with LATENCY.labels(model=model).time():
    try:
        result = generate(prompt)
        REQUESTS.labels(model=model, status="ok").inc()
    except Exception:
        REQUESTS.labels(model=model, status="error").inc()
        raise
```

优点 埋点靠近业务边界，知道模型、状态和完整操作时长。

缺口 还需分别观测排队、Prefill、Decode，并确保异常路径也被计数。

代码审查：本地 LLM 项目的指标草案还隐含哪些生产风险？

```
llm_tokens_total = Counter(  
    'llm_tokens_total', ...,  
    ['model', 'direction'])  
  
llm_cost_usd_total = Counter(  
    'llm_cost_usd_total', ...,  
    ['model', 'customer'])  
  
llm_gpu_memory_used_bytes = Gauge(  
    ..., ['gpu_id', 'model'])
```

审查结论

- ▶ direction=input/output 是有限维度，合理；
- ▶ customer 可能是高基数，需改为 tenant_tier 或在账单系统聚合；
- ▶ gpu_id 需与 cluster/node 资源身份组合；
- ▶ 成本币种/价格版本、tokens/s 更新窗口必须有明确语义。

适用边界 示例代码不是“标准答案”；还要检查基数、重启、单位、窗口和多租户语义。

rate 与 increase：把累计事件变成速率或窗口增量

```
sum by (model) (rate(llm_requests_total[5m]))
```

```
sum by (tool) (increase(tool_calls_total{status="error"}[1h]))
```

窗口太短 样本少、噪声大，抓取抖动可能放大。

窗口太长 变化被平滑，事故发生与恢复确认变慢。

选窗原则 结合抓取周期、事件频率、告警时效和业务周期。

聚合：先明确“保留哪些维度”，再写 `sum by`

```
sum by (model, region) (rate(llm_requests_total{status="error"}[5m]))
```

按实例看 能找到坏副本，
但序列多、总体趋势不清晰。

按模型/区域看 适合路由、容量与发布问题的第一层定位。

全局求和 适合业务总量，
但可能掩盖单区域或单版本故障。

诊断策略 先从用户影响对应的维度切片，再逐步下钻；不要一开始就盯着单 Pod。

histogram_quantile: 先按 le 合并桶，再估计分位数

```
histogram_quantile(0.95,  
    sum by (le, model) (rate(llm_request_duration_seconds_bucket[5m]))  
)
```

1. `_bucket` 是 Counter，先对窗口求 rate；
2. 保留 `le` 标签，因为它表示桶上界；
3. 对同模型的实例桶求和，得到集群级分布；
4. `histogram_quantile` 在桶之间插值估计 P95；
5. 若目标值附近桶太宽，估计误差会很大。

Recording Rules: 把常用重查询预计算成新序列

为什么需要

- ▶ Dashboard 反复计算复杂聚合；
- ▶ 告警查询必须稳定且低延迟；
- ▶ 长时间范围查询代价高；
- ▶ 希望建立统一的 SLI 与业务语义。

示例

```
record: model:request_rate5m
expr: sum by (model) (
    rate(llm_requests_total[5m])
)
```

代价 预计算降低查询成本，但会增加新序列和治理负担；规则应像代码一样评审、测试和版本化。

从查询到告警：必须加入持续时间与上下文

```
alert: LLMHighErrorRate
expr: model:errors_rate5m / model:request_rate5m > 0.02
for: 10m
labels: { severity: "page" }
annotations: { runbook: "...", dashboard: "..." }
```

for 抑制瞬态噪声，但也增加发现延迟。

labels 用于路由与聚合，必须低基数且稳定。

annotations 提供影响、Dashboard、Trace 搜索和 Runbook 入口。

第 5 讲讨论 AI 指标告警，第 6 讲展开持续性与告警治理；本页强调告警是对可观测查询的可操作封装。

代码审查：本地 alerts.yml 的 HighLatency 查询为什么需要修正？

材料中的草案

```
expr: histogram_quantile(0.95,  
    llm_request_duration_seconds_bucket  
) > 5
```

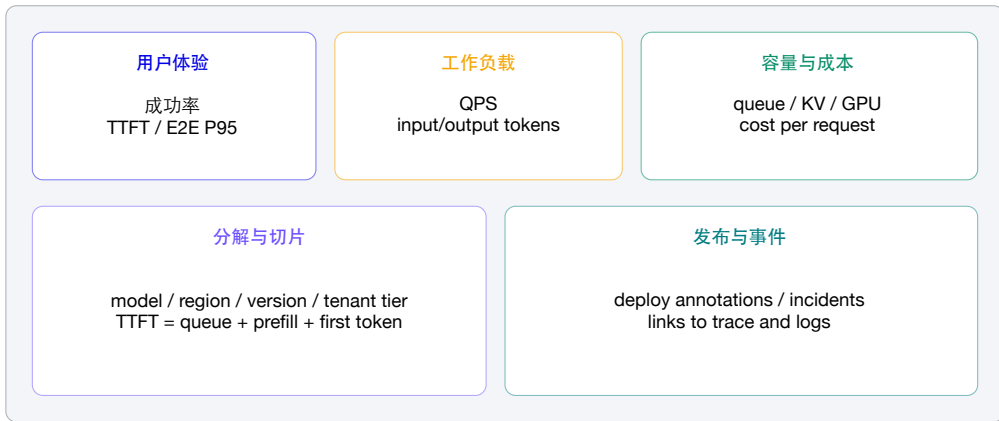
- ▶ bucket 是累计 Counter，不能直接拿当前累计值；
- ▶ 多副本桶未按 1e 正确聚合；
- ▶ 没有保留 model/endpoint，告警定位信息不足。

一种生产化修正

```
expr: histogram_quantile(0.95,  
    sum by (le, model, endpoint) (  
  
    rate(llm_request_duration_seconds_bucket[5m])  
    )  
) > 5
```

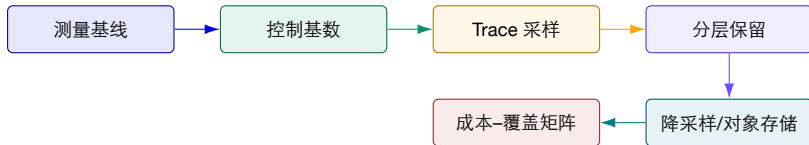
- ▶ 再结合最小请求量，避免低流量抖动；
- ▶ annotation 应链接模型 Dashboard 与慢 Trace。

Dashboard 不是指标仓库，而是面向决策的视图



第一屏回答“用户是否受影响”，第二层解释“哪个维度变化”，再提供到日志和 Trace 的下钻入口。

材料实验：可观测成本优化必须同时写出“省了什么、失去什么”



技术	主要节省	必须记录的覆盖损失
删除 request/user 标签 正常 Trace 仅采样 1% 原始指标 14 天, 5m 聚合 90 天 INFO 仅保留 7 天	Prometheus 活跃序列 Span 存储与网络 长期块存储 日志存储	指标不再支持单请求查询, 需由 Trace 承担 未采样正常请求无法事后下钻; 错误/慢请求应保留 无法对旧事故做秒级重建 较久历史只能依赖聚合与审计日志

练习目标 材料提出 60% 成本下降、覆盖损失低于 5% 的目标；真正的答案是可审计的权衡矩阵，而不是只报节省比例。

四、OpenTelemetry：统一埋点与上下文

它不是另一个 Dashboard，而是一套生成、传播、处理和导出遥测数据的开放标准与工具。

OpenTelemetry 的信号范围正在扩展

Traces 请求
路径与 Span。

Metrics 运
行时测量与聚
合。

Logs 事件
记录与上下
文。

Baggage 跨
边界传播的上
下文。

演进中
Events、
Profiles 等。



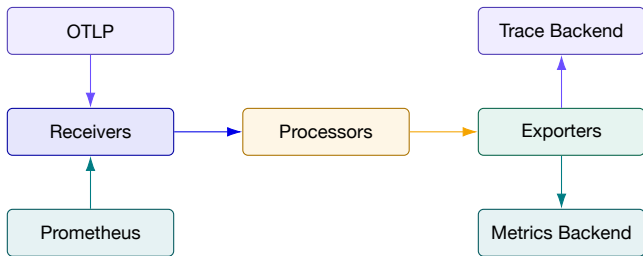
官方当前将 Traces、Metrics、Logs、Baggage 列为支持的信号；Events 与 Profiles 仍在演进。

API、SDK、Collector、Backend：职责不要混在一起

组件	主要职责	设计价值
API SDK Collector Backend OTLP	在代码中创建 Span、Meter、Log 等抽象 采样、聚合、处理、资源信息与导出 接收、处理、路由、批量、重试和导出 存储、索引、查询、展示、告警与分析 遥测数据的标准传输协议	库作者可埋点而不绑定具体后端 应用部署者决定数据如何生成和发送 将后端适配和数据治理从应用进程解耦 可选 Prometheus、Jaeger、Tempo、商业平台等 减少每个应用直接适配多个后端

核心 OpenTelemetry 主要解决 “如何产生和运输一致的遥测数据”，不替代 Prometheus/Grafana/Trace 后端。

Collector Pipeline: Receiver → Processor → Exporter



Receiver 接收 OTLP、Prometheus、Jaeger 等格式。

Processor 批处理、限流、过滤、属性变换、采样。

Exporter 发送到一个或多个存储和分析后端。

自动埋点与手工埋点：覆盖面和业务语义的权衡

Auto-instrumentation

- ▶ 快速覆盖 HTTP、RPC、数据库和常用框架；
- ▶ 适合建立端到端路径的基本骨架；
- ▶ 代码改动少，便于大规模推广；
- ▶ 但不知道“检索命中”“规划步骤”“任务结果”等业务语义。

Manual instrumentation

- ▶ 在关键业务边界创建 Span、指标和事件；
- ▶ 可以记录模型版本、路由、工具和结果；
- ▶ 更能支持假设验证和业务 SLI；
- ▶ 需要代码治理，避免重复、敏感字段和语义漂移。

推荐路径 先自动建立调用骨架，再对最重要的任务阶段手工补充语义，而不是给每个函数都创建 Span。

Resource 与语义约定：先说明“是谁”，再说明“做了什么”

Resource：遥测数据来自谁

- ▶ service.name / service.version;
- ▶ deployment.environment;
- ▶ cloud.region / k8s.cluster.name;
- ▶ host、container、process;
- ▶ 模型服务可补充部署与加速器资源信息。

Span/Metric attributes：操作是什么

- ▶ HTTP/RPC/DB 语义字段;
- ▶ model、operation、tool、status;
- ▶ input/output token 桶或请求类别;
- ▶ 不直接记录密钥、完整 Prompt 和 PII;
- ▶ 控制取值集合与后端索引成本。

语义约定让不同语言、框架和团队产生可组合的数据；自定义字段应有明确命名空间和版本。

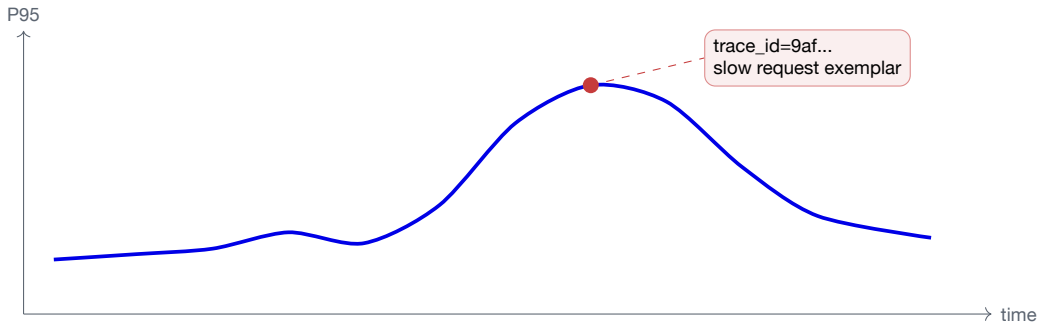
Context Propagation: 跨进程父子关系依赖显式传递



异步队列、批处理、重试、流式响应和 Agent 子任务最容易丢失上下文；需要定义父子关系、link 关系与重试语义。

安全边界 Baggage 会沿调用传播，不应放入敏感信息；跨租户和外部工具边界要过滤。

Exemplar: 从聚合指标跳到代表性 Trace



价值 指标图上的异常点携带代表性 Trace 引用，值班人员不再手工猜测时间范围、服务和请求。

Exemplar 不是保存所有请求；它为聚合分布选择少量可下钻样本。

研究案例：AgentTracer 要解决“语义懂意图、内核只见流量”的割裂

只看语义层

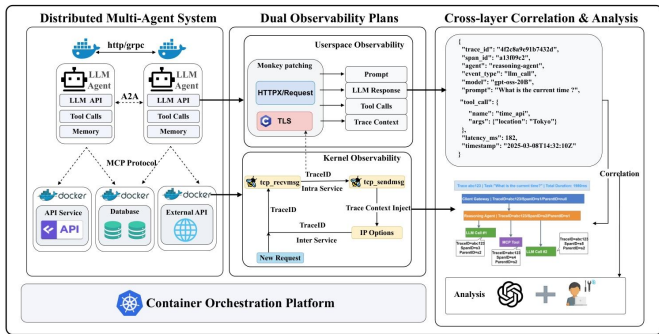
- ▶ 知道 Agent、Prompt、工具与任务意图；
- ▶ 能判断“调用了不存在的工具”；
- ▶ 但难以看到跨容器、跨节点的真实网络与内核执行；
- ▶ 异步并发下，用户空间事件难对应到具体连接。

只看基础设施层

- ▶ 能看到 socket、TCP、进程和跨节点流量；
- ▶ 能发现丢包、重传、带宽和系统调用异常；
- ▶ 但不知道这段流量属于哪个 Agent 决策或工具意图；
- ▶ 网络正常也不能排除工具或参数语义错误。

研究问题 能否在不修改分布式多智能体应用代码的前提下，用同一个 TraceID 确定性关联语义事件与内核事件？

AgentTracer 的双观测平面：共享 TraceID 连接语义与执行



读图顺序

1. 用户空间钩子捕获 LLM 调用、Prompt、工具与 Trace Context;
2. 内核 eBPF 观察 send/recv 等执行事件;
3. TraceID 在服务内和跨节点传播;
4. 两类事件按同一请求关联;
5. 上层分析同时拥有“意图”与“发生了什么”。

两个观测平面分别记录任务行为与系统运行状态，通过请求上下文关联。

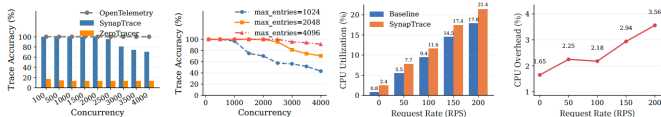
AgentTracer 案例：为什么跨层证据可以排除错误根因？

场景	内核层看到什么	语义层看到什么	诊断结论
Retry Storm	Agent 与 MCP 之间重复请求	每次工具返回触发新的 LLM 决策步骤	Prompt 控制流导致循环，而非网络/工具不可达
Tool Hallucination	多次错误期间没有真实的工具网络流量	LLM 反复选择 Schema 中不存在的工具	错在工具选择，故障发生在 socket 之前
Wrong Parameter	网络往返正常、无异常重传	参数字段与用户意图不一致	错在参数生成逻辑，而非网络或服务端

反证检查 检查反证：如果根因是网络，应当出现哪些观测结果？

关键能力 跨层关联不是增加证据数量，而是让某层的正常行为能排除另一类假设。

阶段性实验怎么读：准确率、并发上限与开销要一起看



不要只看一根柱子

- ▶ Trace accuracy 随并发变化；
- ▶ 上下文表容量不足会造成关联下降；
- ▶ 提高容量可改善准确率，但增加状态成本；
- ▶ CPU 利用与额外开销随请求率上升；
- ▶ 工程目标是准确性、容量与开销的 Pareto 平衡。

这些数字是项目阶段性实验结果，适合用来训练“如何读系统论文图”，不应脱离硬件、并发和配置条件泛化。

五、时序存储与 AIOps 数据采集与存储

可观测数据必须在成本、时效、精度、保留期和查询体验之间取得平衡。

时序数据库的工作负载与普通事务数据库不同

写入特征

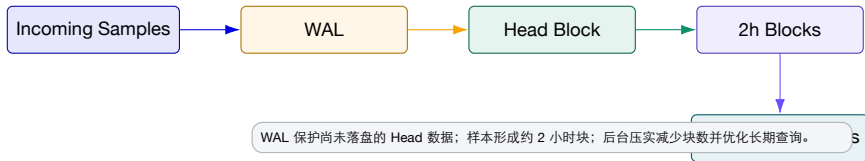
- ▶ 持续追加，时间基本单调；
- ▶ 同一批抓取写入大量序列样本；
- ▶ 高吞吐、少更新、极少随机修改；
- ▶ 标签集合决定索引和序列身份。

查询特征

- ▶ 按标签选择一组序列；
- ▶ 在时间窗口内 rate、sum、quantile；
- ▶ 近期查询多，事故时突发高并发；
- ▶ 长周期趋势需要降采样或预计算。

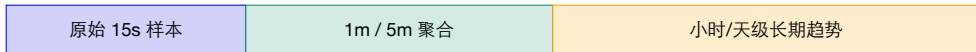
设计结果 写优化的数据布局、压缩、倒排标签索引、时间分块和保留策略成为核心。

Prometheus 本地存储：Head、WAL、Block 与 Compaction



崩溃恢复、磁盘容量、保留期和 compaction I/O 都是 Prometheus 自身需要被监控的对象。

保留期、降采样与记录规则：精度和成本不能同时无限增长



→ 数据年龄

故障下钻
高精度
短保留

容量分析
SLO 报表

季节性
预算与规划
低成本长保留

事故定位需要近期高精度；季度容量规划不需要保留每 15 秒的全部原始序列。

本地、联邦与远程存储：规模扩大后的三种方向

模式	适合场景	主要权衡
单 Prometheus 分片/联邦 Remote Write Sidecar/全局查询	单集群、近期查询、故障域清晰 多集群分层汇总、保留局部自治 长期存储、全局查询、多租户与大规模 保留各集群本地 Prometheus 并提供统一视图	简单；容量、HA 和全局视图有限 需要规划聚合层级，原始数据跨层下钻复杂 网络、队列、重试、乱序和后端成本 组件更多，一致性、对象存储与缓存需治理

企业实践 通常按故障域保留本地自治，再将精选数据远程写入全局平台；不要让一个中央系统成为唯一观测入口。

时序数据库选型：从工作负载与治理要求出发

评估维度	要问的问题	对 AI 平台的影响
规模与基数 查询模式 保留与成本 可用性 多租户与安全 生态与运维	活跃序列数、写入样本/s、标签增长？ 近期排障、全局聚合还是长期趋势？ 原始数据保留多久？是否降采样？ 单集群、跨区、灾备和降级要求？ 隔离、配额、加密、审计如何实现？ PromQL、OTel、对象存储、团队经验？	多模型、多租户、GPU 与 Histogram 放大序列 TTFT 下钻和季度容量分析需求不同 高频 GPU 指标和 token 指标成本显著 事故时观测系统必须仍可查询 Prompt/租户/业务元数据涉及敏感边界 迁移成本和供应商绑定需要评估

“监控监控系统”：遥测管道本身也会失败

需要观察的信号

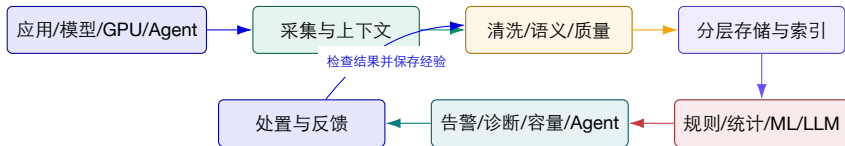
- ▶ scrape up、抓取时长、样本数与丢弃数；
- ▶ Collector 接收、队列、丢弃、重试和导出失败；
- ▶ WAL、磁盘、Head series、compaction；
- ▶ remote write backlog 与延迟；
- ▶ 查询延迟、超时、规则评估失败。

危险的静默失败

- ▶ Dashboard 平滑，不是系统稳定，而是停止采集；
- ▶ 高基数保护丢弃了关键标签；
- ▶ 时钟漂移使跨系统事件顺序错误；
- ▶ Collector 内存保护开始丢数据；
- ▶ 告警规则因查询超时未被正确评估。

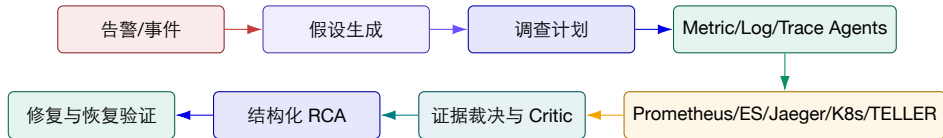
最低要求 为遥测管道建立独立健康视图、数据新鲜度检查和外部黑盒探针。

企业级 AIOps 数据采集与存储：从遥测到决策的流水线



平台价值不止统一存储，更在于统一身份、时间、拓扑、语义、质量、权限和处理结果的记录与反馈。

AgenticSRE 案例：诊断时如何查询指标、日志和调用链？



工具层 提供原始或结构化证据，不直接宣布最终根因。

Agent 层 根据假设选择数据源、时间窗、对象与查询。

裁决层 检查支持、反驳、冲突和缺失，再决定是否闭合。

这张架构的核心不是“大模型接了很多工具”，而是把证据采集、推理、风险和验证分成可审计边界。

多集群可观测性：上下文隔离是证据正确性的前提

隔离键	通算示例	智算示例
cluster_id / domain	social-network / general	vllm-t4 / llm
namespace / workload	otel-demo / frontend	llm-k8s / vllm-server
object identity	Deployment / Service / Pod	Endpoint / Model / GPU / Process
time window	告警前基线-故障-恢复	KV/queue/TTFT 同一活动窗口
data provenance	Prometheus / Jaeger / K8s	GPU exporter / vLLM / TELLER

输入隔离 事件进入 RCA 前先绑定集群、领域、命名空间和对象。

查询隔离 Tool Registry 选择对应端点和凭据，不跨环境混查。

记忆隔离 历史召回按域和对象过滤，避免旧经验污染当前证据。

证据模型：支持、反驳、缺失与冲突应有不同语义

证据类型	对假设的作用	示例
支持证据	提高分支可信度，但要检查来源是否独立	KV Cache 99% + waiting 上升 + TTFT 上升
反驳证据	降低或淘汰分支，通常强于弱支持	CPU 正常且故障解除不随 CPU 变化，反驳 CPU 压力
缺失证据	不能当作反驳，但限制置信度上限	Jaeger 无数据只说明 Trace 证据缺失
冲突证据	触发二次调查并降低自动处置权限	日志指向应用错误，网络对象却与恢复时间吻合

对象一致

时间一致

来源独立

因果可检验

数据可追溯

结论 证据不是越多越好；高质量证据必须能区分候选假设，并实际改变下一步调查或处置。

同一份可观测数据，支持四类企业级能力

能力	需要的数据与方法	输出
告警治理	SLI、基线、依赖、事件、规则执行与历史反馈	去重、抑制、路由、优先级、Runbook
故障诊断	指标切片、日志事件、Trace、拓扑与变更	候选根因、证据链、影响范围
容量分析	负载、利用率、饱和度、排队、成本与季节性	扩缩容、资源配额、模型路由与预算
智能体工具	可查询遥测、CMDB、变更、知识库和执行审计	调查计划、工具调用、解释、受控处置

共同前提 数据有统一语义和时间，能按服务/模型/版本/区域关联，并且每条自动化结论可以回溯到原始证据。

为运维智能体设计“证据型工具”，而不是把 Dashboard 截图交给它

工具接口示例

- ▶ `query_metrics(promql, start, end, step);`
- ▶ `search_logs(filter, trace_id, limit);`
- ▶ `get_trace(trace_id);`
- ▶ `get_topology(service, time);`
- ▶ `get_changes(scope, window)。`

返回结果应包含

- ▶ 查询语句、时间范围、数据新鲜度；
- ▶ 序列/事件/Span 与稳定资源身份；
- ▶ 单位、采样、缺失与截断说明；
- ▶ 数据来源、权限与审计 ID；
- ▶ 可复查的引用，而非只有自然语言总结。

原则 智能体可以生成假设，但事实必须来自可重放的工具调用；处置前要检查权限、评估风险，并约定恢复判定条件。

TELLER 1/7: LLM 推理为何需要比服务 Trace 更深的跨层证据?

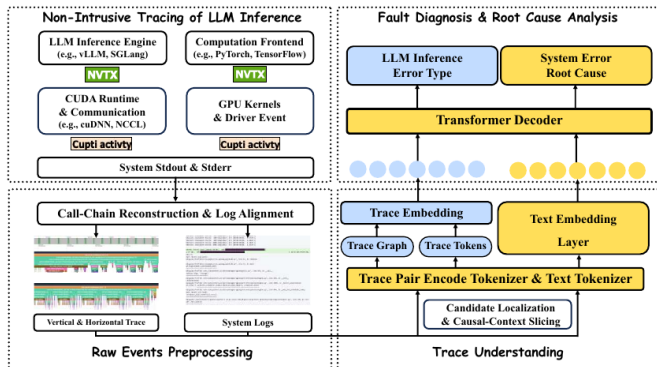
层级	主要职责与信号	可能出现的故障
推理引擎层	请求调度、连续批处理、KV Cache、Prefill/Decode 步骤	排队、抢占、阶段误判、缓存压力
计算后端层	PyTorch/TensorFlow、算子与库调用	算子选择、形状、同步与框架状态
Host Operator 层	CUDA Runtime/Driver、Stream/Event、内存搬运	启动延迟、同步、Driver/Runtime 错误
Device Operator 层	GPU Kernel 与显存操作	Kernel 异常、性能退化、错误状态
横向通信层	NCCL collective、同步与传输	慢 Rank、链路拥塞、collective 停滞

异步 Host 调用与 Device Kernel 不天然在同一时间线上对齐。

并发 多个请求被合并进共享算子和通信组，原始事件难以归属请求。

论文问题 如何在不修改模型二进制和服务逻辑的前提下，把跨层事件重建成请求级、可用于根因分析的证据?

TELLER 2/7: 四步把原始事件变成可解释的根因证据

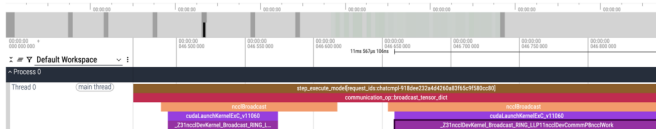


按流水线读图

1. NVTX/CUPTI 非侵入采集；
2. 重建请求调用链并对齐日志；
3. 定位候选算子并切取因果上下文；
4. 压缩 Trace，与文本联合诊断；
5. 输出故障类型、可疑算子和根因解释。

左半边是确定性的系统证据处理，右半边是表示学习与推理；这种分层让原始证据可以被复查，而不是全部交给大模型“自由理解”。

TELLER 3/7: 请求级调用链重建到底在“对齐”什么?



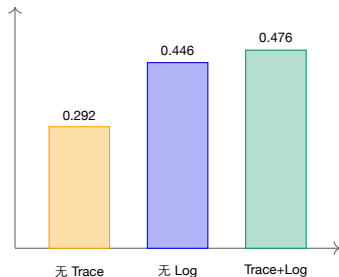
一条请求的层次

- ▶ Engine step 带 request 标识;
- ▶ 前端/后端算子嵌套在 step 中;
- ▶ Host CUDA call 与 GPU Kernel 通过线程、correlation ID 和 launch 关系绑定;
- ▶ NCCL 事件补充横向通信, 日志按 request ID/时间窗口对齐。

形成的证据 一棵保留父子、先后、耗时和通信关系的请求级调用链树。

TELLER 4/7: 为什么“只看日志”与“只让 LLM 读摘要”都不够?

模态消融：Fault Macro F1



完整方法 vs Prompt-only

方法	H Step F1	H Op F1
TELLER	0.916	0.806
Prompt-only	0.668	0.303

- ▶ 日志提供语义，但缺少低层执行结构；
- ▶ Trace 提供结构，却不能独立解释所有错误；
- ▶ 只把摘要扔给 LLM，缺少候选定位和结构保持表示。

实验说明的是“结构化跨层证据有增益”，不是“大模型一定优于传统方法”；结果还受数据集和评测协议约束。

TELLER 5/7: Trace 压缩不是越狠越好——定位依赖细粒度结构

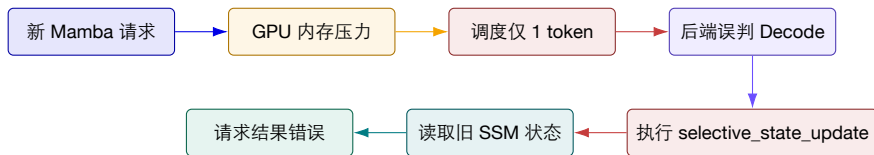
TPE 词表	每步事件	压缩率	H Op F1
原始	2241.92	—	—
256	361.43	83.88%	0.806
512	25.90	98.84%	0.173
1024	3.88	99.83%	0.138

如何解释？

- ▶ TPE 合并重复的结构片段，显著缩短输入；
- ▶ 过度合并会让不同算子/父子关系变得不可区分；
- ▶ 粗粒度语义仍能支持“像什么故障”；
- ▶ 精确定位“哪个算子有问题”更依赖细节；
- ▶ 最优点是压缩与诊断保真度的平衡。

可迁移启示 无论是 Trace 采样、日志摘要还是指标降采样，都应“以目标诊断问题还能否回答”评价，而非只追求更小的数据量。

TELLER 6/7: 一个跨层矛盾如何暴露调度错误?



引擎/日志证据 这是新请求，本应走 Prefill。

设备 Trace 证据 实际启动 Decode Kernel，而正常 Prefill 请求执行另一 Kernel。

根因定位 单层工具只看到各自“合理”的片段；请求级对齐后出现“新请求却执行 Decode”的跨层矛盾，直接指向路由/调度错误。

TELLER 7/7: 系统论文还必须讲清部署成本与结论边界

论文报告的追踪开销

- ▶ Wall time: 64.77s → 71.15s, +9.8%;
- ▶ CPU utilization: 217% → 236%, +8.8%;
- ▶ 最大常驻内存: +3.25%;
- ▶ “非侵入式”表示不改二进制/业务逻辑, 不表示零开销。

需要检查的边界

- ▶ 300 条请求、5 类推理设置、5.63M 跨层节点;
- ▶ 双节点、6 张 A40、Qwen3-8B 为主;
- ▶ 切分是 request-disjoint, 但可能共享 workload/run;
- ▶ 更大集群、长期多租户和跨工作负载泛化仍需验证。

阅读方法 先理解机制与证据贡献, 再检查数据分布、基线、公平性、开销和外部有效性; 这才是把论文转化为工程知识。

综合练习：为 AI 运维助手设计可观测方案

系统

- ▶ 用户问题 → RAG → LLM 规划；
- ▶ 调用 CMDB、Prometheus、日志、Kubernetes；
- ▶ 高风险动作需要人工审批；
- ▶ 目标：5 分钟内提出可验证故障假设。

小组产出：一页设计

1. 5 个关键指标及其类型/标签；
2. 一条 Agent Trace 的 Span 树；
3. 3 类必须记录的结构化事件；
4. Prometheus/OTel/存储数据流；
5. 一项高基数或敏感数据控制；
6. 智能体结论如何引用证据并被验证。

评价：覆盖业务结果、模型/工具行为和资源状态；数据能关联；成本与安全可控；结论可追溯。

桶计数算例：能否判定 95% 的请求在 1 秒内完成？

某窗口无重启、无漏采；两个副本采用相同的经典直方图桶。表中数字为窗口内累计桶增量。

副本	不超过 0.5 秒	不超过 1 秒	总请求数
A	50	90	100
B	700	900	900
合并	750	990	1,000

$$1 \text{ 秒内比例} = \frac{90 + 900}{100 + 900} = 99\%$$

推导与判断 该窗口满足 95% 的门槛。先合并同边界桶再计算；不能平均两个副本的 P95。仅凭这些桶也无法得到精确 P95，只能定位区间并作插值估计。

算例使用假设数据。

进一步分析

1. 如果只能保留 7 天原始指标和 90 天聚合指标，你会如何分层？
2. 为什么 request_id 适合日志/Trace，却通常不适合指标标签？
3. 某模型 TTFT P95 高，但 GPU 利用率只有 45%，你会提出哪些假设？
4. 自动埋点已经覆盖所有 HTTP/RPC，为什么仍然无法判断 RAG 答案质量？
5. 如果运维智能体查询到的数据存在 10 分钟延迟，它是否可以自动扩容或重启？

下一讲：结构化日志、分布式追踪与跨信号故障诊断

参考资料与延伸阅读

- ▶ Prometheus Documentation: Data Model、Metric Types、Querying、Storage、Recording Rules;
- ▶ OpenTelemetry Documentation: Signals、Context Propagation、Collector、Semantic Conventions;
- ▶ Google, *Site Reliability Engineering*: Monitoring Distributed Systems;
- ▶ Brendan Gregg, USE Method; Tom Wilkie, RED Method;
- ▶ Xu et al., *TELLER*, ASE 2026;
- ▶ 本地课程材料: `ai-infra-engineer-learning-main/lessons/mod-108-monitoring-observability/`;
- ▶ 课程工程材料: LLM deployment metrics、alerting rules 与 LLM observability exercise。

官方链接: prometheus.io/docs/concepts/data_model; prometheus.io/docs/prometheus/latest/storage;
opentelemetry.io/docs/concepts/signals; opentelemetry.io/docs/collector

让系统留下证据

让工程师与智能体基于同一事实行动

校企联合研究生课程 面向 AI 原生系统的智能运维